

Q2. Distance Matrix Computation with k-NN Classification

Objectives:

- Check **missing values** and impute using **PDF (Probability Distribution Function)** to choose central tendency.
- **Standardize** features.
- Compute pairwise distance matrices using **Euclidean** and **Manhattan** via `sklearn.metrics.pairwise_distances`.
- Use the distance matrix with **k-Nearest Neighbors** classification.
- Apply **cross-validation** with a **heuristic (elbow) method** to find optimal `k`.
- Report the best **F1 score** and **Precision**.

Environment: `uv` with scikit-learn, numpy, pandas, matplotlib, seaborn.

```
In [ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path
from scipy.stats import skew
from sklearn.metrics.pairwise import pairwise_distances
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import make_scorer, precision_score

print("numpy:", np.__version__)
print("pandas:", pd.__version__)
print("seaborn:", sns.__version__)
```

1. Load Dataset & Check Missing Values

```
In [ ]: BASE_DIR = Path(".")
df = pd.read_excel(BASE_DIR / "lab2 data.xlsx")

print(f"Original shape: {df.shape}")
print(f"Columns: {df.columns.tolist()}\n")

# Check missing values
missing = df.isnull().sum()
missing_pct = (missing / len(df)) * 100
missing_df = pd.DataFrame({'Missing Count': missing, 'Missing %': missing_pct})
missing_df = missing_df[missing_df['Missing Count'] > 0].sort_values('Missing Count')

if missing_df.empty:
    print("No missing values found in any column.")
else:
    print("Columns with missing values:")
    display(missing_df)
```

2. Select Features, Clean & Check Missing Within Selected Features

```
In [ ]: # Select feature columns and target
n_features = df[['Rate your contribution towards extra curricular activities',
                  'Rate your technical competencies',
                  'your CIA % of last semester',
                  'Your maximum attendance % till last semester']]

target = df['Internships Interests'].copy()

# --- Clean CIA, Attendance ---
def clean_pct(col):
    vals = pd.to_numeric(col, errors='coerce')
    mask = vals <= 1
    vals[mask] = vals[mask] * 100
    return vals

n_features['CIA_pct'] = clean_pct(n_features['your CIA % of last semester'])
n_features['Attend_pct'] = clean_pct(n_features['Your maximum attendance % till last semester'])

feature_cols = [
    'Rate your contribution towards extra curricular activities',
    'Rate your technical competencies',
    'CIA_pct',
    'Attend_pct'
]

# Rename for convenience
rename_map = {
    'Rate your contribution towards extra curricular activities': 'ExtraCurricular',
    'Rate your technical competencies': 'TechRating',
}

df_work = n_features[feature_cols].rename(columns=rename_map)
work_cols = ['ExtraCurricular', 'TechRating', 'CIA_pct', 'Attend_pct']

print("Feature matrix shape:", df_work.shape)
print(f"\nMissing values in selected features:\n{df_work.isnull().sum()}")
```

3. PDF-Based Imputation of Central Tendencies

For each numeric feature, we plot its **Probability Distribution Function (PDF)** (via histogram + and measure **skewness** to decide the imputation strategy:

- **Symmetric / near-zero skew** (~ -0.5 to 0.5) → **Mean** imputation
- **Skewed** ($\text{lskewl} > 0.5$) → **Median** imputation (robust to outliers)
- **Discrete / categorical-like** → **Mode** imputation

```
In [ ]: fig, axes = plt.subplots(2, 2, figsize=(12, 8))
        axes = axes.flatten()

        impute_strategies = {}

        for idx, col in enumerate(work_cols):
            ax = axes[idx]
            series = df_work[col].dropna()

            # Plot PDF: histogram + KDE
            sns.histplot(series, kde=True, bins=20, ax=ax, color='#2E86AB', alpha=0.3)

            # Compute skewness
            s = skew(series)

            # Decide strategy
            unique_vals = series.nunique()
            if unique_vals <= 5:
                strategy = 'mode'
                impute_val = series.mode().iloc[0]
            elif abs(s) <= 0.5:
                strategy = 'mean'
                impute_val = series.mean()
            else:
                strategy = 'median'
                impute_val = series.median()

            impute_strategies[col] = {'strategy': strategy, 'value': round(impute_val, 2)}

            ax.axvline(impute_val, color='red', linestyle='--', linewidth=2, label=f'Impute Value')
            ax.set_title(f"{col} | skew={s:.3f} → {strategy}")
            ax.legend()
            ax.grid(True, alpha=0.3)

        plt.tight_layout()
        plt.show()

        print("Imputation strategies based on PDF analysis:")
        for col, info in impute_strategies.items():
            print(f"  {col:20s} skew={info['skew']:.3f} → {info['strategy']}: {info['value']:.2f}")
```

Apply the chosen imputation

```
In [ ]: df_imputed = df_work.copy()

        for col in work_cols:
```

```

info = impute_strategies[col]
missing_before = df_imputed[col].isnull().sum()

if info['strategy'] == 'mode':
    val = df_imputed[col].mode().iloc[0]
elif info['strategy'] == 'median':
    val = df_imputed[col].median()
else:
    val = df_imputed[col].mean()

df_imputed[col] = df_imputed[col].fillna(val)

if missing_before > 0:
    print(f"{col:20s} filled {missing_before} missing values with {i

print(f"\nMissing after imputation:\n{df_imputed.isnull().sum()}")

```

4. Prepare Target & Encode

```

In [ ]: # Combine with target and drop any remaining NaN rows
valid = target.notna()
X_raw = df_imputed[valid].astype(float)
y_raw = target[valid]

le = LabelEncoder()
y_clean = le.fit_transform(y_raw)

print(f"Samples: {len(X_raw)}")
print(f"Target distribution:\n{pd.Series(y_clean).value_counts()}")
print(f"Class mapping: {dict(zip(le.classes_, le.transform(le.classes_)))}")
print(f"\nFirst 5 rows:\n{X_raw.head()}")

```

5. Standardize Features

Distance-based algorithms are sensitive to feature scales. We apply Z-score standardization.

```

In [ ]: scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_raw)

print("Post-standardization:")
print("  means:", X_scaled.mean(axis=0).round(6))
print("  stds: ", X_scaled.std(axis=0).round(6))

```

6. Compute Pairwise Distance Matrices

Using `sklearn.metrics.pairwise_distances` with two metrics.

```

In [ ]: D_euclidean = pairwise_distances(X_scaled, metric='euclidean')
D_manhattan = pairwise_distances(X_scaled, metric='manhattan')

print(f"Euclidean distance matrix shape: {D_euclidean.shape}")
print(f"Manhattan distance matrix shape: {D_manhattan.shape}")
print(f"\nEuclidean distances (first 5x5):")
print(np.round(D_euclidean[:5, :5], 3))

```

```
print(f"\nManhattan distances (first 5x5):")
print(np.round(D_manhattan[:5, :5], 3))
```

7. k-NN with Cross-Validation & Heuristic (Elbow) Method

Train k-NN using **precomputed distance matrices**. Evaluate $k=1..30$ with 5-fold **Stratified CV**, **F1 (macro)** and **Precision (macro)** to find the best k via the elbow heuristic.

```
In [ ]: k_range = range(1, 31)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

results = []

for metric_name, D in [('Euclidean', D_euclidean), ('Manhattan', D_manhattan)]:
    f1_scores = []
    prec_scores = []

    for k in k_range:
        knn = KNeighborsClassifier(n_neighbors=k, metric='precomputed')

        f1 = cross_val_score(knn, D, y_clean, cv=cv, scoring='f1_macro')
        prec = cross_val_score(knn, D, y_clean, cv=cv,
                               scoring=make_scorer(precision_score, average='macro'))

        f1_scores.append(f1.mean())
        prec_scores.append(prec.mean())

        results.append({
            'Distance Metric': metric_name,
            'k': k,
            'F1 (macro)': round(f1.mean(), 4),
            'Precision (macro)': round(prec.mean(), 4)
        })

    best_idx = np.argmax(f1_scores)
    best_k = k_range[best_idx]
    print(f"[{metric_name}] Best k = {best_k} | F1 = {f1_scores[best_idx]}, Precision = {prec_scores[best_idx]}")

results_df = pd.DataFrame(results)
print("\nAll results:")
display(results_df)
```

8. Heuristic (Elbow) Plot

The elbow heuristic looks for the k where the F1 score plateaus.

```
In [ ]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for idx, metric_name in enumerate(['Euclidean', 'Manhattan']):
    subset = results_df[results_df['Distance Metric'] == metric_name]
    ax = axes[idx]

    ax.plot(subset['k'], subset['F1 (macro)'], 'o-', label='F1 (macro)',
            ax.plot(subset['k'], subset['Precision (macro)'], 's--', label='Precision (macro)')

    best_row = subset.loc[subset['F1 (macro)'].idxmax()]
    best_k = best_row['k']
    print(f"Best k for {metric_name} is {best_k}")
```

```

ax.axvline(best_row['k'], color='gray', linestyle=':', alpha=0.7)
ax.annotate(f"k={int(best_row['k'])}",
            xy=(best_row['k'], best_row['F1 (macro)']),
            xytext=(5, 10), textcoords='offset points',
            fontsize=10, fontweight='bold', color='#A23B72')

ax.set_xlabel('k (number of neighbors)')
ax.set_ylabel('Score')
ax.set_title(f'{metric_name} Distance – Elbow Plot')
ax.legend()
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

9. Best Result Summary

```

In [ ]: print("=== Best Configuration for Each Distance Metric ===\n")

for metric_name in ['Euclidean', 'Manhattan']:
    subset = results_df[results_df['Distance Metric'] == metric_name]
    best = subset.loc[subset['F1 (macro)'].idxmax()]
    print(f"{metric_name}:")
    print(f"  Optimal k (elbow): {int(best['k'])}")
    print(f"  F1 Score (macro): {best['F1 (macro)']}")
    print(f"  Precision (macro): {best['Precision (macro)']}")
    print()

global_best = results_df.loc[results_df['F1 (macro)'].idxmax()]
print(f"*** Global Best ***")
print(f"  Distance: {global_best['Distance Metric']}")
print(f"  k = {int(global_best['k'])}")
print(f"  F1 (macro) = {global_best['F1 (macro)']}")
print(f"  Precision (macro) = {global_best['Precision (macro)']}")

```

Pipeline Summary

Step	Detail
1. Missing Value Check	Count and % of nulls per column
2. PDF Imputation	Plot distribution + KDE; measure skew; choose mean/median/mode
3. Standardization	Z-score scaling (mean=0, std=1)
4. Distance Matrices	pairwise_distances with Euclidean & Manhattan
5. k-NN CV (5-fold)	k=1..30 with precomputed distance
6. Elbow Heuristic	Plateau detection on F1 vs k plot
7. Best Metrics	F1 (macro) & Precision (macro) reported